COL100: Introduction to Computer Science
Lab 5: Recursion

February 3, 2025

1 Recursion

Recursion is a powerful programming technique where a function calls itself in order to solve a problem. This tutorial presents the concept of recursion in python with examples. A recursive function typically has the following structure:

1. Base case(s): This handles the smallest possible version of the problem and provides a direct answer.

2. Recursive case(s): This breaks down the problem into smaller versions and calls the function itself.

The factorial of a number n (denoted as $n!$) is the product of all positive integers up to n. Using recursion, the factorial can be defined as:

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \times (n-1)! & \text{otherwise} \end{cases}$$

Below is the code to compute the factorial of a number using recursion:

```
def factorial(int n):
    if (n == 0):
        return 1
    else:
        return n * factorial(n - 1)

num = int(input("Enter a number: "))
fact = factorial(num)
print(f""Factorial of {num} = {fact}")
```

2 Submission Problem

2.1 Instructions

- Complete the code of the following problem in **main.py** program file.
- Submit the program file in the assignment titled "Lab 5" on Gradescope.
- The Autograder on Gradescope will evaluate the submitted program on some test cases.

2.2 Count Ways to Reach Top of the Staircase

You are at the bottom of a staircase with N stairs. You can climb the stairs in one of the following ways:

- Take 1 step at a time.
- Take 2 steps at a time.
- Take 3 steps at a time.

Your task is to determine the number of ways you can reach the top of the staircase, i.e., the Nth stair, after starting from the bottom.

Sample Input

An integer representing the total number of stairs.

```
4
```

Sample Output

An integer representing the number of distinct ways to reach the top of staircase.

```
7
```

Explanation

There are 7 different ways to reach the 4th stair:

- 1 step + 1 step + 1 step + 1 step
- 1 step + 1 step + 2 steps
- 1 step + 2 steps + 1 step

- 2 steps + 1 step + 1 step
- 2 steps + 2 steps
- 1 step + 3 steps
- 3 steps + 1 step

3 Practice Problems

3.1 Power of a Number using Recursion

You need to implement a recursive function to compute the power of a number, i.e., a^b. The problem of computing the power of a number can be modelled as a recursive problem by using the properties of exponents. Specifically:

$$a^b = a \times a^{b-1}$$

The base case can be when $b = 0$, such that $a^0 = 1$ for any value of a.

Requirements

1. Implement a function that takes a float value a and an integer exponent value b as inputs, and returns the float value a^b correct up to two decimal places.
2. Your function should use recursion.

Sample Input

```
2.0
3
```

Sample Output

```
8.00
```

Optimizations

- Can you think why this way of defining the recursive formulation of Exponentiation is wasteful ?
- Is there a better way of defining Exponentiation recursively?
- Hint:

$$a^b = \begin{cases} a^{\lfloor b/2 \rfloor} \times a^{\lfloor b/2 \rfloor} & \text{if } b \text{ is even} \\ a^{\lfloor b/2 \rfloor} \times a^{\lfloor b/2 \rfloor} \times a & \text{if } b \text{ is odd} \end{cases}$$

- Count the number of recursive calls made in the two cases.

3.2 Greatest Common Divisor

You need to implement a recursive function to compute the greatest common divisor (GCD) of two integers.

The greatest common divisor (GCD) of two numbers a and b is the largest number that divides both of them without leaving a remainder. The Euclidean algorithm can be used to find the GCD of two numbers using recursion.

The algorithm is based on the principle that the GCD of two numbers also divides their difference. To find the GCD of a and b, we subtract the smaller number from the larger number. The GCD of a and b is the same as the GCD of the larger number and the difference between the two.

The recursive formula to represent this is:

$$\text{GCD}(a, b) = \begin{cases} a & \text{if } b = 0 \\ \text{GCD}(b, a \mod b) & \text{otherwise} \end{cases}$$

Sample Input 1

```
56 98
```

Sample Output 1

```
14
```

Sample Input 2

```
36 12
```

Sample Output 2

```
12
```

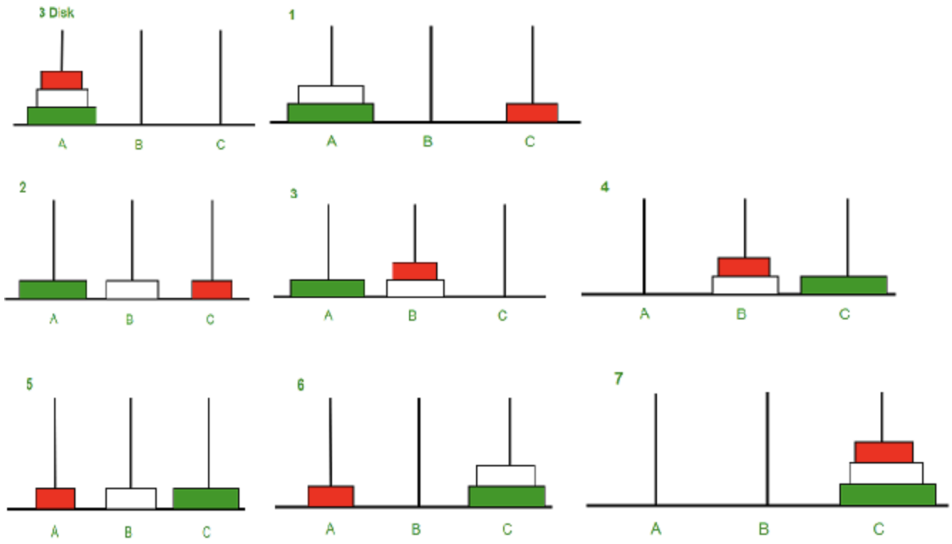
3.3 Tower of Hanoi

The Tower of Hanoi is a classic problem in which you have three towers ('A', 'B', and 'C') with one tower containing a stack of ('n') disks. The objective is to move the entire stack of (n) disks from one tower (A) to another tower (C) obeying the following rules.

1. Only one disk can be moved at a time.
2. Each move consists of taking the top disk from one of the stacks and placing it on top of another stack.
3. No disk may be placed on top of a smaller disk.

Instructions:

1. Write a program with function `towerOfHanoi(n, from_tower, to_tower, aux_tower)` that prints the sequence of moves to solve the Tower of Hanoi problem.
2. Your program should take an integer as input (the number of disks) and print the steps required to solve the problem.



Sample Input 1

```
2
```

Sample Output 1

```
Move disk 1 from tower A to tower B
Move disk 2 from tower A to tower C
Move disk 1 from tower B to tower C
```

Sample Input 2

```
3
```

Sample Output 2

```
Move disk 1 from tower A to tower C
Move disk 2 from tower A to tower B
Move disk 1 from tower C to tower B
Move disk 3 from tower A to tower C
Move disk 1 from tower B to tower A
Move disk 2 from tower B to tower C
Move disk 1 from tower A to tower C
```